

MUP Lab — Memory System in Verilog

Complete Solutions with Explanations

This document contains the full Verilog implementation for all modules in the Memory System lab, following the circuit diagram provided in class. Each module is presented with a brief explanation of the design decisions, followed by the complete, commented Verilog source.

Important: The ROM contents (initial block values) in `rom32byte` are placeholder examples. Replace them with the values specified in your actual lab sheet / test output waveforms.

1. D_ff (Provided — no marks)

The D flip-flop is the fundamental storage primitive. Everything clocks on the **positive edge** only. Two conditions are checked in priority order: reset first, then write.

Reset is *synchronous* — it only clears the output at a clock edge, not immediately. The write condition requires all three enables: `chipSelect`, `enable` (from the decoder), and `regWrite`. If none of these conditions match, the implicit behaviour of `always @(posedge clk)` is to hold the value — no `else` branch is needed.

■ Non-blocking assignment (`<=`) is used inside the clocked `always` block — this is mandatory for correct sequential behaviour.

■ This module is provided in the lab and does not need to be modified.

D_ff.v

```
module D_ff(
    input  clk,
    input  chipSelect,
    input  reset,
    input  regWrite,
    input  enable,
    input  d,
    output reg q
);
    // All behaviour is clocked – no asynchronous paths
    always @(posedge clk) begin
        if (chipSelect == 1'b1 && reset == 1'b1)
            q <= 1'b0; // synchronous reset (highest priority)
        else if (chipSelect == 1'b1 && enable == 1'b1 && regWrite == 1'b1)
            q <= d; // capture input on write
        // else: q holds its current value (implicit register behaviour)
    end
endmodule
```

2. mux2to1_16bits (1 mark)

A simple 16-bit wide 2-to-1 multiplexer. When `select = 0`, the output is `in0`; when `select = 1`, the output is `in1`.

This mux is used at the output stage of every memory block. `in0` is always tied to `16'b0`, so when `outputEnable` or `chipSelect` is low, the memory drives zero onto the bus rather than floating.

■ *The sensitivity list uses all three inputs (`in0`, `in1`, `select`) — this ensures the output updates whenever any input changes, making it correctly combinational.*

■ *Using always `@(*)` is equivalent and also acceptable.*

`mux2to1_16bits.v`

```
// If select = 0 -> muxOut = in0
// If select = 1 -> muxOut = in1
module mux2to1_16bits(
    input  [15:0] in0,
    input  [15:0] in1,
    input          select,
    output reg [15:0] muxOut
);
    always @(in0, in1, select) begin
        case (select)
            1'b0: muxOut = in0;
            1'b1: muxOut = in1;
        endcase
    end
endmodule
```

3. decoder5to32 (2 marks)

A 5-to-32 decoder takes a 5-bit binary input and asserts exactly one of 32 output lines — this is called a **one-hot** encoding. It is used at two levels:

First, inside `ram32byte` / `rom32byte` to select which of the 32 registers/words to enable for writing.

Second, inside `ram1kb` / `rom1kb` to select which of the 32 sub-blocks to activate.

The `decoder4to16` follows an identical pattern with 4-bit input and 16-bit output, and is shown for completeness.

■ *Alternative one-liner: assign `decOut = (32'd1 << selectIn)`; — functionally identical.*

■ *Always include a default case to avoid inferred latches in synthesis.*

`decoder5to32.v`

```
module decoder5to32(
    input  [4:0] selectIn,
    output reg [31:0] decOut
);
    // One-hot: exactly one bit asserted for each input value
    always @(selectIn) begin
        case (selectIn)
```

```
5'd0: decOut = 32'h0000_0001;
5'd1: decOut = 32'h0000_0002;
5'd2: decOut = 32'h0000_0004;
5'd3: decOut = 32'h0000_0008;
5'd4: decOut = 32'h0000_0010;
5'd5: decOut = 32'h0000_0020;
5'd6: decOut = 32'h0000_0040;
5'd7: decOut = 32'h0000_0080;
5'd8: decOut = 32'h0000_0100;
5'd9: decOut = 32'h0000_0200;
5'd10: decOut = 32'h0000_0400;
5'd11: decOut = 32'h0000_0800;
5'd12: decOut = 32'h0000_1000;
5'd13: decOut = 32'h0000_2000;
5'd14: decOut = 32'h0000_4000;
5'd15: decOut = 32'h0000_8000;
5'd16: decOut = 32'h0001_0000;
5'd17: decOut = 32'h0002_0000;
5'd18: decOut = 32'h0004_0000;
5'd19: decOut = 32'h0008_0000;
5'd20: decOut = 32'h0010_0000;
5'd21: decOut = 32'h0020_0000;
5'd22: decOut = 32'h0040_0000;
5'd23: decOut = 32'h0080_0000;
5'd24: decOut = 32'h0100_0000;
5'd25: decOut = 32'h0200_0000;
5'd26: decOut = 32'h0400_0000;
5'd27: decOut = 32'h0800_0000;
5'd28: decOut = 32'h1000_0000;
5'd29: decOut = 32'h2000_0000;
5'd30: decOut = 32'h4000_0000;
5'd31: decOut = 32'h8000_0000;
default: decOut = 32'h0;
endcase
end
endmodule
```

4. register16bit (2 marks)

A 16-bit register is simply 16 D flip-flops wired in parallel, sharing all control signals (clk, chipSelect, reset, regWrite, enable) and individually connected to one bit of inR / outR.

The **generate block** is the key construct here. It elaborates the loop at compile time, creating 16 distinct D_ff instances with the index *i* used to connect to bit *i* of the buses. The label BIT_FF is required to give each instance a unique hierarchical name.

- The enable line is the SAME for all 16 flip-flops — it comes from the decoder and selects the entire register at once.

- genvar *i* must be declared outside the generate block in Verilog-2001 syntax.

- Named port connections (.clk(clk)) are strongly preferred over positional to avoid wiring mistakes.

register16bit.v

```
`include "D_ff.v"
module register16bit(
    input  clk,
    input  chipSelect,
    input  reset,
    input  regWrite,
    input  enable,          // driven by decoder – selects THIS register
    input  [15:0] inR,      // 16-bit data to write
    output [15:0] outR      // 16-bit data output (always visible)
);
    // Instantiate 16 D flip-flops, one per bit
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : BIT_FF
            D_ff dff_inst (
                .clk      (clk),
                .chipSelect (chipSelect),
                .reset     (reset),
                .regWrite  (regWrite),
                .enable    (enable),    // same enable for all bits in the register
                .d         (inR[i]),
                .q         (outR[i])
            );
        end
    endgenerate
endmodule
```

5. mux32to1 (2 marks)

Selects one 16-bit word from 32 register outputs based on a 5-bit address. This is the read path of the RAM — all register outputs are always visible, and this mux picks the one that corresponds to the requested address.

The mux is purely combinational — output updates immediately when select or any input changes. always @(*) ensures the sensitivity list is automatically complete.

■ In the previous lab (Lab 9), the equivalent module was `mux32to2` which selected TWO bytes (one 16-bit word from pairs of 8-bit registers). Here, since each register is already 16 bits wide, the output is a single 16-bit word per address.

■ The default case outputs `16'b0` to avoid simulation unknowns (X) if select is ever out of range.

`mux32to1.v`

```
module mux32to1(
    input  [15:0] in0,  in1,  in2,  in3,
    input  [15:0] in4,  in5,  in6,  in7,
    input  [15:0] in8,  in9,  in10, in11,
    input  [15:0] in12, in13, in14, in15,
    input  [15:0] in16, in17, in18, in19,
    input  [15:0] in20, in21, in22, in23,
    input  [15:0] in24, in25, in26, in27,
    input  [15:0] in28, in29, in30, in31,
    input  [4:0]  select,
    output reg [15:0] muxOut
);
    always @(*) begin
        case (select)
            5'd0: muxOut = in0;   5'd1: muxOut = in1;
            5'd2: muxOut = in2;   5'd3: muxOut = in3;
            5'd4: muxOut = in4;   5'd5: muxOut = in5;
            5'd6: muxOut = in6;   5'd7: muxOut = in7;
            5'd8: muxOut = in8;   5'd9: muxOut = in9;
            5'd10: muxOut = in10; 5'd11: muxOut = in11;
            5'd12: muxOut = in12; 5'd13: muxOut = in13;
            5'd14: muxOut = in14; 5'd15: muxOut = in15;
            5'd16: muxOut = in16; 5'd17: muxOut = in17;
            5'd18: muxOut = in18; 5'd19: muxOut = in19;
            5'd20: muxOut = in20; 5'd21: muxOut = in21;
            5'd22: muxOut = in22; 5'd23: muxOut = in23;
            5'd24: muxOut = in24; 5'd25: muxOut = in25;
            5'd26: muxOut = in26; 5'd27: muxOut = in27;
            5'd28: muxOut = in28; 5'd29: muxOut = in29;
            5'd30: muxOut = in30; 5'd31: muxOut = in31;
            default: muxOut = 16'b0;
        endcase
    end
endmodule
```

6. ram32byte (3 marks)

This is the core RAM building block, wiring together all the previously defined modules into a functioning 32-word (64-byte) memory. The data flow is:

Write path (synchronous): Address $\rightarrow$ decoder5to32 $\rightarrow$ one-hot enable $\rightarrow$ selected register16bit captures writeData at next posedge clk (if chipSelect and writeEnable are high).

Read path (asynchronous/combinational): All 32 register outputs are fed into mux32to1 → selected output passes through mux2to1_16bits (gated by outputEnable & chipSelect) → memOut.

Reads are *always combinational*. Only writes are edge-triggered. This is standard SRAM behaviour.

■ The generate block inside ram32byte instantiates 32 register16bit modules — a generate inside a module that itself uses generate. This is perfectly valid Verilog.

■ Notice the output gate: `mux2to1_16bits` with `in0=16'b0` ensures `memOut` is driven to zero (not floating) when the chip is not selected or not reading.

■ The same address wire drives BOTH the decoder (for writes) and the mux (for reads). This is correct — the decoder selects which register to enable for writing, and the mux selects which register to present for reading.

ram32byte.v

```
include "decoder5to32.v"
`include "register16bit.v"
`include "mux32to1.v"
`include "mux2to1_16bits.v"

module ram32byte(
    input clk,
    input chipSelect,
    input reset,
    input writeEnable,
    input outputEnable,
    input [4:0] address,      // 5-bit: selects 1 of 32 registers
    input [15:0] writeData,
    output [15:0] memOut
);
// ■■ 1. Decode address → one-hot write enable ████████████████████████████████████
wire [31:0] decOut;
decoder5to32 writeDec (
    .selectIn (address),
    .decOut   (decOut)
);
// ■■ 2. Storage: 32 × 16-bit registers ████████████████████████████████████████████
wire [15:0] Dmem [0:31];     // outputs of all 32 registers
genvar i;
generate
    for (i = 0; i < 32; i = i + 1) begin : RAM_REG
        register16bit reg_i (
            .clk          (clk),
            .chipSelect   (chipSelect),
            .reset         (reset),
            .regWrite      (writeEnable),
```

```

        .enable      (decOut[i]), // only the addressed register's enable is 1
        .inR         (writeData),
        .outR        (Dmem[i])

    );

end
endgenerate

// ■■ 3. Read mux: select the addressed register's output ■■■■■■■■■■■■■■■■■■■■■■
wire [15:0] memReadOut;
mux32to1 readMux (
    .in0(Dmem[0]),    .in1(Dmem[1]),    .in2(Dmem[2]),    .in3(Dmem[3]),
    .in4(Dmem[4]),    .in5(Dmem[5]),    .in6(Dmem[6]),    .in7(Dmem[7]),
    .in8(Dmem[8]),    .in9(Dmem[9]),    .in10(Dmem[10]), .in11(Dmem[11]),
    .in12(Dmem[12]), .in13(Dmem[13]), .in14(Dmem[14]), .in15(Dmem[15]),
    .in16(Dmem[16]), .in17(Dmem[17]), .in18(Dmem[18]), .in19(Dmem[19]),
    .in20(Dmem[20]), .in21(Dmem[21]), .in22(Dmem[22]), .in23(Dmem[23]),
    .in24(Dmem[24]), .in25(Dmem[25]), .in26(Dmem[26]), .in27(Dmem[27]),
    .in28(Dmem[28]), .in29(Dmem[29]), .in30(Dmem[30]), .in31(Dmem[31]),
    .select  (address),
    .muxOut  (memReadOut)
);

// ■■ 4. Output gate: drive 0 when OE or CS is not asserted ■■■■■■■■■■■■■■■■■■■■■■
mux2to1_16bits outGate (
    .in0    (16'b0),
    .in1    (memReadOut),
    .select (outputEnable & chipSelect), // both must be 1 to read
    .muxOut (memOut)
);

endmodule

```

7. rom32byte (2 marks)

A ROM has no write port — contents are fixed at design/synthesis time. The `initial` block pre-loads a register array at simulation start. The read is a simple conditional assign: when both `chipSelect` and `outputEnable` are high, the selected address is output; otherwise zero.

There is **no clock** in a ROM — all reads are purely combinational. This is the key structural difference from `ram32byte`.

■ Replace the initial block values with those specified in your lab's expected waveforms or test file.

■ The ternary assign (condition `? a : b`) is equivalent to the `mux2to1` pattern used in RAM, but more concise since there is no structural hierarchy to maintain.

■ In synthesis, an initial block will be mapped to ROM/LUT contents. In simulation, it initialises memory at time 0.

rom32byte.v

```
module rom32byte(
    input  chipSelect,
    input  outputEnable,
    input  [4:0]  address,
    output [15:0] dataOut
);
    // Fixed contents – pre-loaded at elaboration time
    reg [15:0] mem [0:31];
    initial begin
        // Example ROM contents (replace with lab-specified values)
        mem[0]  = 16'hDEAD; mem[1]  = 16'hBEEF;
        mem[2]  = 16'h1234; mem[3]  = 16'h5678;
        mem[4]  = 16'hABCD; mem[5]  = 16'hEF01;
        mem[6]  = 16'h0000; mem[7]  = 16'hFFFF;
        mem[8]  = 16'hCAFE; mem[9]  = 16'hBABE;
        mem[10] = 16'h0001; mem[11] = 16'h0002;
        mem[12] = 16'h0004; mem[13] = 16'h0008;
        mem[14] = 16'h0010; mem[15] = 16'h0020;
        mem[16] = 16'h0040; mem[17] = 16'h0080;
        mem[18] = 16'h0100; mem[19] = 16'h0200;
        mem[20] = 16'h0400; mem[21] = 16'h0800;
        mem[22] = 16'h1000; mem[23] = 16'h2000;
        mem[24] = 16'h4000; mem[25] = 16'h8000;
        mem[26] = 16'hAAAA; mem[27] = 16'h5555;
        mem[28] = 16'hF0F0; mem[29] = 16'h0F0F;
        mem[30] = 16'hFF00; mem[31] = 16'h00FF;
    end
    // Combinational read – no clock needed for ROM
    assign dataOut = (chipSelect && outputEnable) ? mem[address] : 16'h0000;
endmodule
```

8. ram1kb (3 marks)

Thirty-two ram32byte blocks are stacked using the same structural pattern: a decoder5to32 on the **upper 5 address bits** (addr[9:5]) selects which block is active. The **lower 5 bits** (addr[4:0]) are passed to all blocks as the within-block address.

Each block's `chipSelect` is the AND of the parent chip select and the decoded block enable — this ensures that if `ram1kb` itself is not selected, no block inside it will activate.

The output mux collects all block outputs and selects the active one. Since only one block has `chipSelect` asserted, only one block drives a non-zero output — the mux just routes that to the final output.

■ The cascaded chipSelect pattern (chipSelect & blockSel[i]) is the standard technique for hierarchical memory. It propagates the 'am I selected?' question down each level.

■ The output mux uses `address[9:5]` as its select — the same upper bits that the decoder used. This consistency is important: the same block that was written to is the one whose output is selected.

ram1kb.v

[illegible]

```
    endgenerate  
// ■■ 3. Output mux: pick the active block's output ■■■■■■■■■■■■■■■■■■■■■  
mux32to1 outMux (  
    .in0(blockOut[0]),   .in1(blockOut[1]),   .in2(blockOut[2]),   .in3(blockOut[3]),  
    .in4(blockOut[4]),   .in5(blockOut[5]),   .in6(blockOut[6]),   .in7(blockOut[7]),  
    .in8(blockOut[8]),   .in9(blockOut[9]),   .in10(blockOut[10]), .in11(blockOut[11]),  
    .in12(blockOut[12]), .in13(blockOut[13]), .in14(blockOut[14]), .in15(blockOut[15]),  
    .in16(blockOut[16]), .in17(blockOut[17]), .in18(blockOut[18]), .in19(blockOut[19]),  
    .in20(blockOut[20]), .in21(blockOut[21]), .in22(blockOut[22]), .in23(blockOut[23]),  
    .in24(blockOut[24]), .in25(blockOut[25]), .in26(blockOut[26]), .in27(blockOut[27]),  
    .in28(blockOut[28]), .in29(blockOut[29]), .in30(blockOut[30]), .in31(blockOut[31]),  
    .select (address[9:5]),  
    .muxOut (memOut)  
);  
endmodule
```

9. rom1kb (2 marks)

Structurally identical to `ram1kb` but using `rom32byte` blocks instead of `ram32byte`. There is no clock, no `writeEnable`, no `writeData` — the only inputs are `address`, `chipSelect`, and `outputEnable`.

The address decode and output mux logic is exactly the same as in ram1kb.

■ `rom1kb` has no `clk` or `reset` ports — it is entirely combinational. Do not add them; it would change the module interface.

rom1kb.v

[illegible]

```
    );
endmodule
```

[illegible]

